Introductory Programming and Problem-Solving Skills

Week - 8

ERROR HANDLING

Basanta Singh

Let’s Recap

- Storage Technologies
- Text Files
- Fundamental Operation of Text Files
- String Processing and String Traversing

Motivation

- **[Scenario]: Imagine you want to send Rs. 5000 to your friend using mobile banking. After adding all the credentials and required information you press the send button. But as soon as you press the send the button a message displays.**

>>>Insufficient Fund

Now, think about:

I. **Why and How Did That Message Displayed?**

II. **What If That Message Was Not Displayed?**

Exception Handling

Introduction

Navigation

1. **Introduction**

- **Various error messages can occur when executing Python programs**. **Such errors are called exceptions**. So far we have let Python handle these errors by reporting them on screen.
- **Exception can be "caught" and "handled" by a program**, however, we can either **correct the error and continue** the execution or **terminate** the program gracefully.

In today's lecture we will be discussing what is exception and how to handle it?

What is an Exception?

Exception

Navigation

1. Exception

- **Exception** [**Definition**]: An exception is a value (object) that is **raised ("thrown")** signaling that an unexpected, or "**exceptional,**" situation has occurred.

- **Python contains a predefined set of exceptions referred to as standard exception**. Further we will see some of the standard exceptions.

Standard Python Exception

Navigation

1. **Exception**

1.1 Standard Python Exception

ImportError	raised when an import (or from…import) statement fails
IndexError	raised when a sequence index is out of range
NameError	raised when a local or global name is not found
TypeError	raised when an operation or function is applied to an object of inappropriate type
ValueError	raised when a built-in operation or function is applied to an appropriate type, but of inappropriate value
IOError	raised when an input/output operation fails (e.g., “file not found”)

Note: The **standard exceptions** are defined within the **exceptions module** of the **Python Standard Library**, which is automatically imported into Python programs

Remember These Errors?

Navigation

1. Exception

1.1 Standard Python Exception

1.2 Some error that occurred.

```
>>> lst = [1, 2, 3]
```

```
>>> lst[3]
Traceback (most recent call last):
  File "<pyshell#1>", line 1, in <module>
    lst[3]
IndexError: list index out of range
```

```
>>> lsst[0]
Traceback (most recent call last):
  File "<pyshell#3>", line 1, in <module>
    lsst[0]
NameError: list index out of range
```

```
>>> 2 + '3'
Traceback (most recent call last):
  File "<pyshell#2>", line 1, in <module>
    2 + '3'
TypeError: unsupported operand type(s) for +: 'int' and 'str'
```

```
>>> int('abc')
Traceback (most recent call last):
  File "<pyshell#4>", line 1, in <module>
    int('abc')
ValueError: invalid literal for int() with base 10: 'abc'
```

- We have seen a number of these exceptions before in our programming.

Let’s Understand Through Example

Navigation

1. Exception

1.1 Standard Python Exception

1.2 Some error that occurred.

Example

What will happen if I pass negative number in the factorial function of the python math module?

```
import math

num = int(input("Enter the number to compute factorial of: "))
print("The factorial of", num ,"ais", math.factorial(num) )
```

Here, when -5 is passed to factorial function, an exception is raised, thrown back to the client code to catch and handle.

```
Week11FileHandling
"E:\Herald Teaching Resources\Introductory - 2023\Practice Code\venv\Scripts\python.exe" "E:\Herald Teaching Resources\Intro
Enter the number to compute factorial of: -5
Traceback (most recent call last):
  File "E:\Herald Teaching Resources\Introductory - 2023\Practice Code\venv\Week11FileHandling.py", line 4, in <module>
    print("The factorial of", num ,"is", math.factorial(num) )
ValueError: factorial() not defined for negative values
```

Note: It raises a Value Error exception when a negative value is passed to it.

Visualization

Navigation

1. Exception

1.1 Standard Python Exception

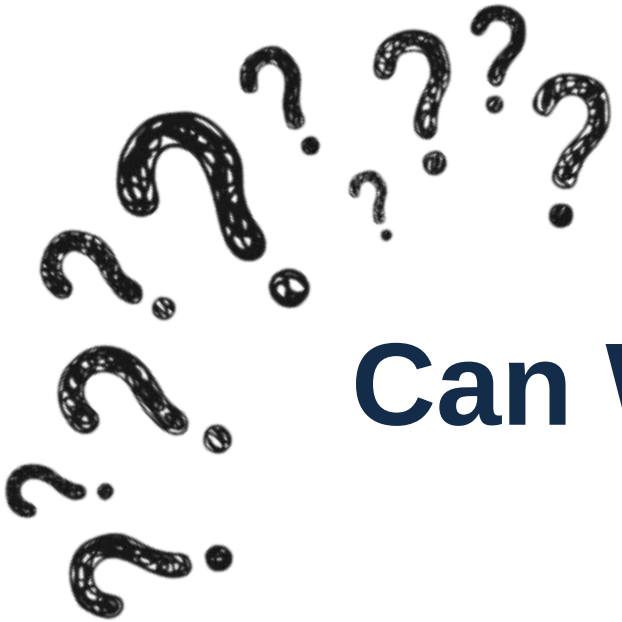
1.2 Some error that occurred.

Visualization


```
"E:\Herald Teaching Resources\Introductory - 2023\Practice Code\venv\Scripts\python.exe" "E:\Herald Teaching Resources\Introdu
Enter the number to compute factorial of: -5
Traceback (most recent call last):
  File "E:\Herald Teaching Resources\Introductory - 2023\Practice Code\venv\Week11FileHandling.py", line 4, in <module>
    print("The factorial of", num ,"is", math.factorial(num) )
ValueError: factorial() not defined for negative values
```

When -5 is passed to the factorial function, an exception is raised, thrown to the client code to catch and handle. Since the client code (in the main module) does not attempt to catch the exception, the exception is caught by the Python Interpreter, causing the program to terminate and display an error message indicating the exception type.

Note: It raises a Value Error exception when a negative value is passed to it.



Can We Handle Such Error?

Exception Handling

Navigation

1. Exception

1.1 Standard Python Exception

1.2 Some error that occurred.

2. Exception Handling

- **Raising and Exception** [**Definition**]: Raising an exception is a way for a function to inform its client a problem has occurred that the function itself cannot handle.

- **[Example]:** suppose a function called getYN prompts a user to enter **'y' or 'n'.** If the user enters something other than these two values, the function **should simple prompt the user again**

- On the other hand, if a function called **isEven** is to be passed a numeric value, but is passed a string instead, **it cannot correct the problem**. It can only **notify the client** of the problem, leaving it up to the client to determine what to do.

Why To Raise Exceptions?

Navigation

1. 1. Exception

1.1 Standard Python Exception

1.2 Some error that occurred.

2. Exception Handling

2.1 Propagation of Exception handling

- Raised exceptions are not **required to be handled** in Python. When an exception is raised and not handled by client code, it is automatically propagated back to the clients calling code. (and it's calling code, etc.) until handled.

- If an exception is thrown all the way back to the **top level** (main module) and **not handled**, then the program terminates and displays the details of the exception.

```
import math
a = -5
print(math.factorial(a))
print("The number cannot be in negative")
a = 5
print(math.factorial(a))
```

```
Traceback (most recent call last):
  File "C:\python\asdfdf.py", line 3, in <module>
    print(math.factorial(a))
ValueError: factorial() not defined for negative values
```

```
import math
try:
    a = -5
    print(math.factorial(a))
except:
    print("The number cannot be in negative")
a = 5
print(math.factorial(a))
```

```
The number cannot be in negative
120
```

try and except

But Before Than That, Imagine, you went to buy shoes, but you are not sure about which shoes to buy and even you are confused about your foot size. So what would you do?

TRY

Fine?

TRY ANOTHER

Try and Except

Navigation

1. Exception

1.1 Standard Python Exception

1.2 Some error that occurred.

2. Exception Handling

2.1 Propagation of Exception handling

2.1 Try and Except

- **Try and Except statement is used to handle these errors with our code in Python**

- **Try** [Definition]: The **try block** is used to **check some code for errors** i.e. the code inside the try block will execute when there is no error in the program.

- **Except** [Definition]: Whereas the code inside the **except block** will execute whenever the program encounters **some error** in the preceding try block.

Let's Fix it

Navigation

1. Exception
1.1 Standard Python Exception
1.2 Some error that occurred.
2. Exception Handling
2.1 Propagation of Exception handling
2.1 Try and Except

- Previously, we were trying to find the factorial of -5 and an exception was thrown as a Value Error and the program was terminated.

```
import math

try:

    num = int(input("Enter the number to com
    print("The factorial of", num, "is", math.fac

except ValueError:

    print("Cannot compute the factorial of neg
```

Here, the call to the factorial function is contained within a **try suite** (**try block**)—the block of code surrounded by try and except headers. The suite following the except header is referred to as an **exception handler.** This exception header "catches" exceptions of type **ValueError**.

Although Exception was handled but our block of code was terminated/skipped

Try, Except and Re-Execution

Navigation

1. Exception
1.1 Standard Python Exception
1.2 Some error that occurred.
2. Exception Handling
2.1 Propagation of Exception handling
2.1 Try and Except
2.2 Try, Except and Re-Execution


```
Enter the number to compute factorial of: -5
Cannot compute the factorial of negative numbers

Process finished with exit code 0
```

This is the output of our previous program where we can see that our program did catch an error while passing negative value. After that our program was terminated.

- **What if we want to re-execute our program if exception is thrown?**
- **Remember the concept, when we used to handle user's input error with while statement?**

Let's Understand Through Example

Navigation

1. Exception

1.1 Standard Python Exception

1.2 Some error that occurred.

2. Exception Handling

2.1 Propagation of Exception handling

2.1 Try and Except

2.2 Try, Except and Re-Execution

Example


```python
import math

num = int(input("Enter the number to compute factorial of: "))
valid_input = False

while not valid_input:
    try:

        print("The factorial of", num, "is", math.factorial(num))
        valid_input = True
    except:
        print("Cannot compute the factorial of negative numbers")
        num = int(input("Enter the number to compute factorial of: "))
```

Week11FileHandling

```
"E:\Herald Teaching Resources\Introductory - 2023\Practice Code\venv\Sc
Enter the number to compute factorial of: -5
Cannot compute the factorial of negative numbers
Enter the number to compute factorial of: 5
The factorial of 5 is 120
```

Here, we can see that program is being recovered from the exception rather than terminating the program, the user is prompted again for input.

Besides exception raised by built-in functions, programmer-defined functions may raise exceptions as well.

Exception Handling and User Input

Navigation

2. Exception Handling

2.1 Propagation of Exception handling

2.1 Try and Except

2.2 Try, Except and Re-Execution

2.3 Exception Handling User Input

- **Suppose we prompted the user to enter the current month as a number,**

```
month = input('Enter current month (1-12): ')
```

- **The input function will return whatever is entered as a string. We can do integer type conversion on this value to make it an integer type:**

```
month = int(input('Enter current month (1-12): '))
```

- **If the input string contains non-digit characters (except for + and -), the int conversion will raise ValueError exception. However what if user enter 14?**

Exception Handling and User Input

Navigation

2. Exception Handling

2.1 Propagation of Exception handling

2.1 Try and Except

2.2 Try, Except and Re-Execution

2.3 Exception Handling User Input


```
valid = False

while not valid:
    try:
        month = int(input('Enter current month (1-12): '))

        while month < 1 or month > 12:
            print('Invalid Input - Must be in the range 1-12'
            month = int(input('Enter current month (1-12): '))
        valid = True
    except ValueError:
        print('Invalid Month Value')
```

It does solves our problem but still it is not an elegant piece of code.

- A better approach is to **design a function** called **getMonth()** that raises a **ValueError** exception for either error condition – if the user enters non digit or, if the user enters a numeric value outside the range 1 – 12

BUT WHY?

Why to Handling Error With Function

Navigation

2. Exception Handling

2.1 Propagation of Exception handling

2.1 Try and Except

2.2 Try, Except and Re-Execution

2.3 Exception Handling User Input

- **Error handling with functions is generally considered better than direct code inside try-except blocks because of:**
- [**Code Reusability**]: When you have a code inside a function, you can reuse that code across multiple parts of your application. This saves you from having to write the same error handling code over and over again
- [**Readability**]: Functions with well-defined error handling blocks make your code more readable. You can clearly see what exception a function is handling and how is it handling
- [**Separation of Concerns**]: By separating error handling code into functions, you keep it to separate from business logic of your code which makes it easier for you to focus on core functionality.

Raising an Exception with Function

Navigation

2. Exception Handling

2.1 Propagation of Exception handling

2.1 Try and Except

2.2 Try, Except and Re-Execution

2.3 Exception Handling with Function


```
def getMonth():
    month = int(input('Enter current month (1-12): '))

    if month < 1 or month > 12:
        raise ValueError('Invalid Month Value')

    return month
```

```
>>> getMonth()
Enter current month (1-12): 1a
Traceback (most recent call last):
  File "<pyshell#18>", line 1, in <module>
    getMonth()
  File " C:\My Python Programs\exception /Raising Excpt Test.py",
line 2, in getMonth
    month = int(input('Enter current month (1-12): '))
ValueError: invalid literal for int() with base 10: '1a'
```

Here, if non digit characters are entered, the ValueError exception is automatically raised by the int type conversion function.

Exception Handling with Function

Navigation

2. Exception Handling

2.1 Propagation of Exception handling

2.1 Try and Except

2.2 Try, Except and Re-Execution

2.3 Exception Handling with Function


```
>>> getMonth()
Enter current month (1-12): 14
Traceback (most recent call last):
  File "<pyshell#23>", line 1, in <module>
    getMonth()
  File " C:\My Python Programs\exception /Raising Excpt Test.py",
line 5, in getMonth
    raise ValueError('Invalid Month Value')
ValueError: Invalid Month Value
```

Here, if non digit characters are entered, the ValueError exception is automatically raised by the int type conversion function.

Note that raised exceptions can optionally have a description that further describes the error, as shown in the figure

```
raise ValueError('Invalid Month Value')
```

Exception Handling with Function

Navigation

2. Exception Handling

2.1 Propagation of Exception handling

2.1 Try and Except

2.2 Try, Except and Re-Execution

2.3 Exception Handling with Function

- **The code looks cleaner but both error displays same message.**

```
valid = False

while not valid:
    try:
        month = getMonth()
        valid = True
    except ValueError:
        print('Invalid Month Value\n')
```

```
Enter current month (1-12): 1a
Invalid Month Value

Enter current month (1-12): 14
Invalid Month Value
```

Here, we can see that error message for the user is much cleaner. However, for both of the error, same message is being displayed (Invalid month)

Raising Exception For Particular Errors

Exception Handling as err_mesg

Navigation

2. Exception Handling

2.1 Propagation of Exception handling

2.1 Try and Except

2.2 Try, Except and Re-Execution

2.3 Exception Handling with Function


```
valid = False

while not valid:
    try:
        month = getMonth()
        valid = True
    except ValueError as err_mesg:
        print(err_mesg,'\n')
```

```
Enter current month (1-12): 1a
invalid literal for int() with base 10: '1a'

Enter current month (1-12): 14
Invalid Month Value

Enter current month (1-12): 12
>>>
```

The line except ValueError as err_messg not only catches the ValueError exceptions, but with as identifier added (here as err_mesg), the error message of the particular ValueError exception is assigned to the specified identifier name.

Exception Handling With File Handling

Navigation

2. Exception Handling

2.1 Propagation of Exception handling

2.1 Try and Except

2.2 Try, Except and Re-Execution

2.3 Exception Handling with Function

2.4 Exception Handling with File Handling

- We saw that when opening a file for reading, an exception is raised if the file cannot be found. In this case, the standard **IOError** exception is raised and the program terminates with a "**No such file directory**" error message. We can catch this exception and handle such an error.

Note: when reading from a text file, the **readline method does not raise any exceptions**. When the end of file is reached, readline returns an empty string rather than throwing an exception, making the code much easier to write.

Let’s understand Through Example

Navigation

2. Exception Handling

2.1 Propagation of Exception handling

2.1 Try and Except

2.2 Try, Except and Re-Execution

2.3 Exception Handling with Function

2.4 Exception Handling with File Handling

Example


```
file_name = input("Enter The File Name")
input_file_opened = False

while not input_file_opened:
    try:
        input_file_opened = open(file_name, 'r')
        print(input_file_opened.read())
        input_file_opened = True

    except IOError:

        print("File Open Error")
        file_name = input("Enter The File Name")
```

Here, we can see that user input file is being opened and read. If the file name is found than try block will be executed. However if the file is not found then IO Error Exception is thrown.

Can We Raise Multiple Exception?

Catching Multiple Exceptions

Navigation

2. Exception Handling

2.1 Propagation of Exception handling

2.1 Try and Except

2.2 Try, Except and Re-Execution

2.3 Exception Handling with Function

2.4 Exception Handling with File Handling

2.4 Catching Multiple Exceptions


```
try:
    f = open('myfile.txt')
    s = f.readline()
    i = int(s.strip())
except (OSError, ValueError, RuntimeError) as
err:
    print("Error: {0}".format(err))
```

```
try:
    f = open('myfile.txt')
    s = f.readline()
    i = int(s.strip())
except OSError as err:
    print("OS Error: {0}".format(err))
except ValueError as err:
    print("Value Error: {0}".format(err))
except RuntimeError as err:
    print("Runtime Error: {0}".format(err))
```

Before Tutorial!!

- Find at least 10 different errors
- Have some research on raising exception, Base exception, Unspecified Exception, Exception Clean ups finally
- What to do when no exceptions occurred

Click me!

https://docs.python.org/3/tutorial/errors.html

Click me!

UNIVERSITY OF WOLVERHAMPTON

BIRATNAGAR INTERNATIONAL COLLEGE

ing